

파이썬을 이용한 데이터 과학 (Data Science using Python)

데이터 정제 및 준비

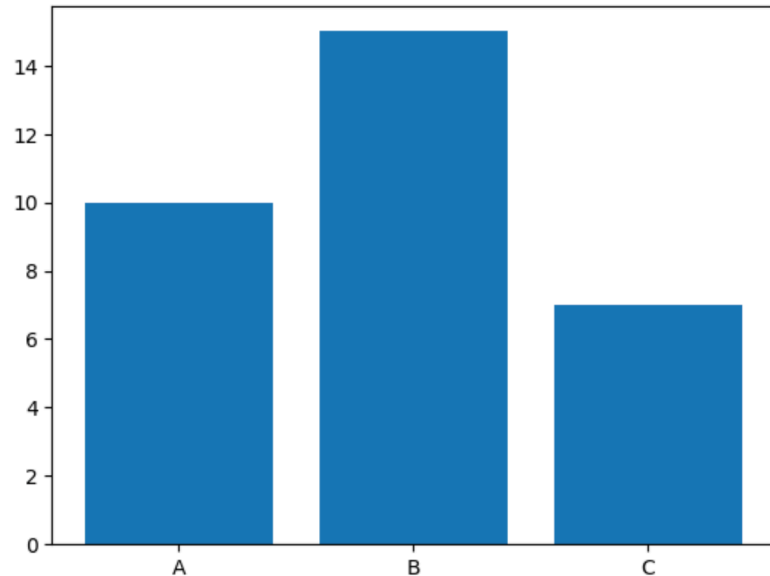


담당교수: 박명훈 (parc.seoultech@seoultech.ac.kr)

Seaborn

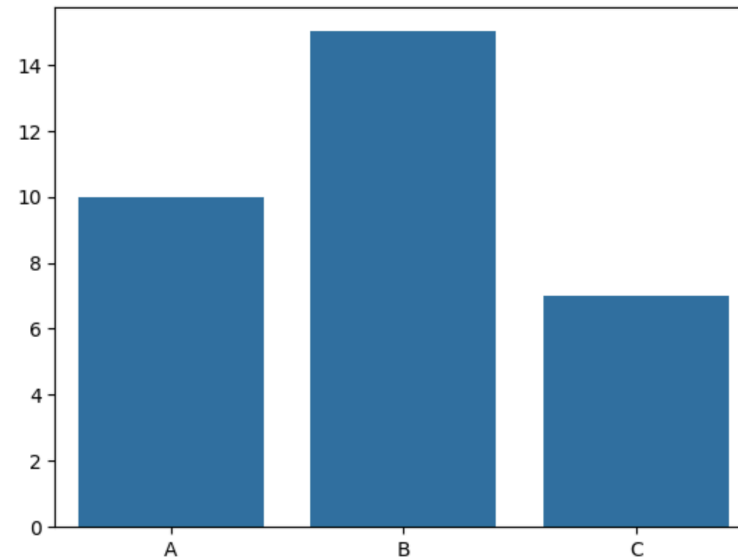
- Matplotlib의 기능과 스타일을 확장한 파이썬 시각화 도구
 - DataFrame과 연동 : pandas.DataFrame을 직접 넘겨서, x='col1', y='col2' 처럼 사용 가능
 - 통계적 시각화 지원
 - 복잡한 plot을 쉽게 그려줌 .
- Seaborn의 간단한 API 로 빠르게 시각화 가능
 - Matplotlib의 정밀한 조정으로, 세밀한 커스터마이징 추가.

```
import matplotlib.pyplot as plt
plt.bar(['A', 'B', 'C'], [10, 15, 7])
plt.show()
```



```
import seaborn as sns
sns.barplot(x=['A', 'B', 'C'], y=[10, 15, 7])
```

<Axes: >



1. 예제 데이터셋 불러오기

```
tips = sns.load_dataset("tips")
```

2. 상위 5개 행 미리보기

```
print(tips.head())
```

3. 전체 행/열 크기 확인

```
print("\nShape:", tips.shape) # (행 수, 열 수)
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Shape: (244, 7)

4. 열 이름과 데이터 타입 확인

```
print("\nColumn Info:")
```

```
print(tips.info())
```

Column Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 244 entries, 0 to 243

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	total_bill	244 non-null	float64
1	tip	244 non-null	float64
2	sex	244 non-null	category
3	smoker	244 non-null	category
4	day	244 non-null	category
5	time	244 non-null	category
6	size	244 non-null	int64

dtypes: category(4), float64(2), int64(1)

memory usage: 7.4 KB

None

5. 기술 통계 요약 (수치형 변수)

```
print("\nDescriptive Statistics:")
```

```
print(tips.describe())
```

Descriptive Statistics:

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

```
# 6. Seaborn barplot 그리기
```

```
plt.figure(figsize=(8, 5))
```

```
sns.barplot(data=tips, x="day", y="tip", hue="day", estimator='mean',  
            palette='pastel', legend=False)
```

```
plt.title("Average Tip by Day", fontsize=16)
```

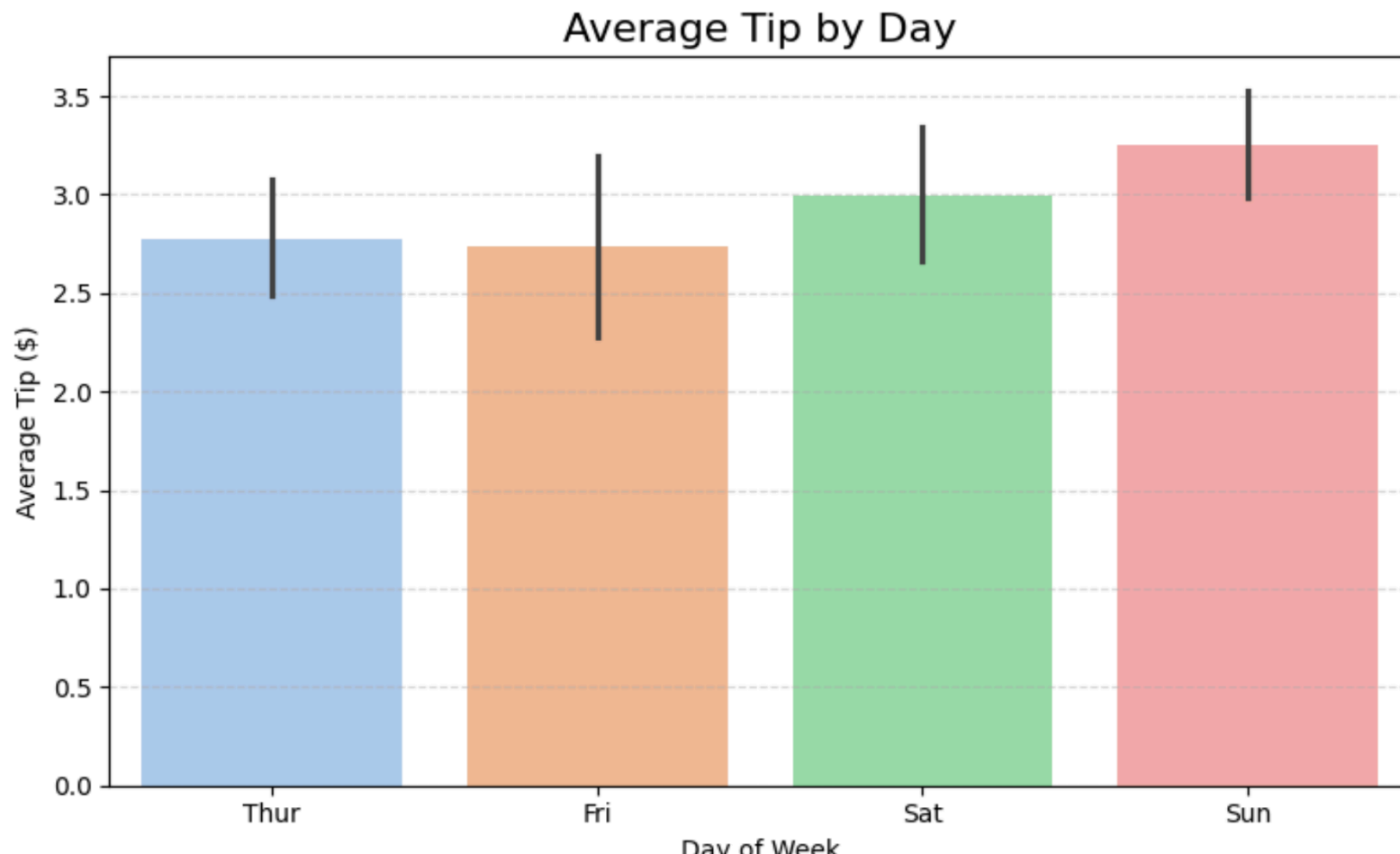
```
plt.xlabel("Day of Week")
```

```
plt.ylabel("Average Tip ($)")
```

```
plt.grid(axis='y', linestyle='--', alpha=0.5)
```

```
plt.tight_layout()
```

```
plt.show()
```



- sns.load_dataset 의 데이터는 Pandas DataFrame

```
type(tips)
```

```
pandas.core.frame.DataFrame
```

```
tips[tips['sex'] == 'Female'] # 여성만
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
11	35.26	5.00	Female	No	Sun	Dinner	4
14	14.83	3.02	Female	No	Sun	Dinner	2
16	10.33	1.67	Female	No	Sun	Dinner	3
...	...	...	...	...	...	...	...
226	10.09	2.00	Female	Yes	Fri	Lunch	2
229	22.12	2.88	Female	Yes	Sat	Dinner	2
238	35.83	4.67	Female	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

87 rows x 7 columns

```
# 요일별 성별 평균 팁 시각화
```

```
plt.figure(figsize=(8, 5))
```

```
sns.barplot(data=tips, x='day', y='tip', hue='sex', estimator='mean', palette='pastel')
```

```
plt.title('Average Tip by Day and Gender')
```

```
plt.xlabel('Day of Week')
```

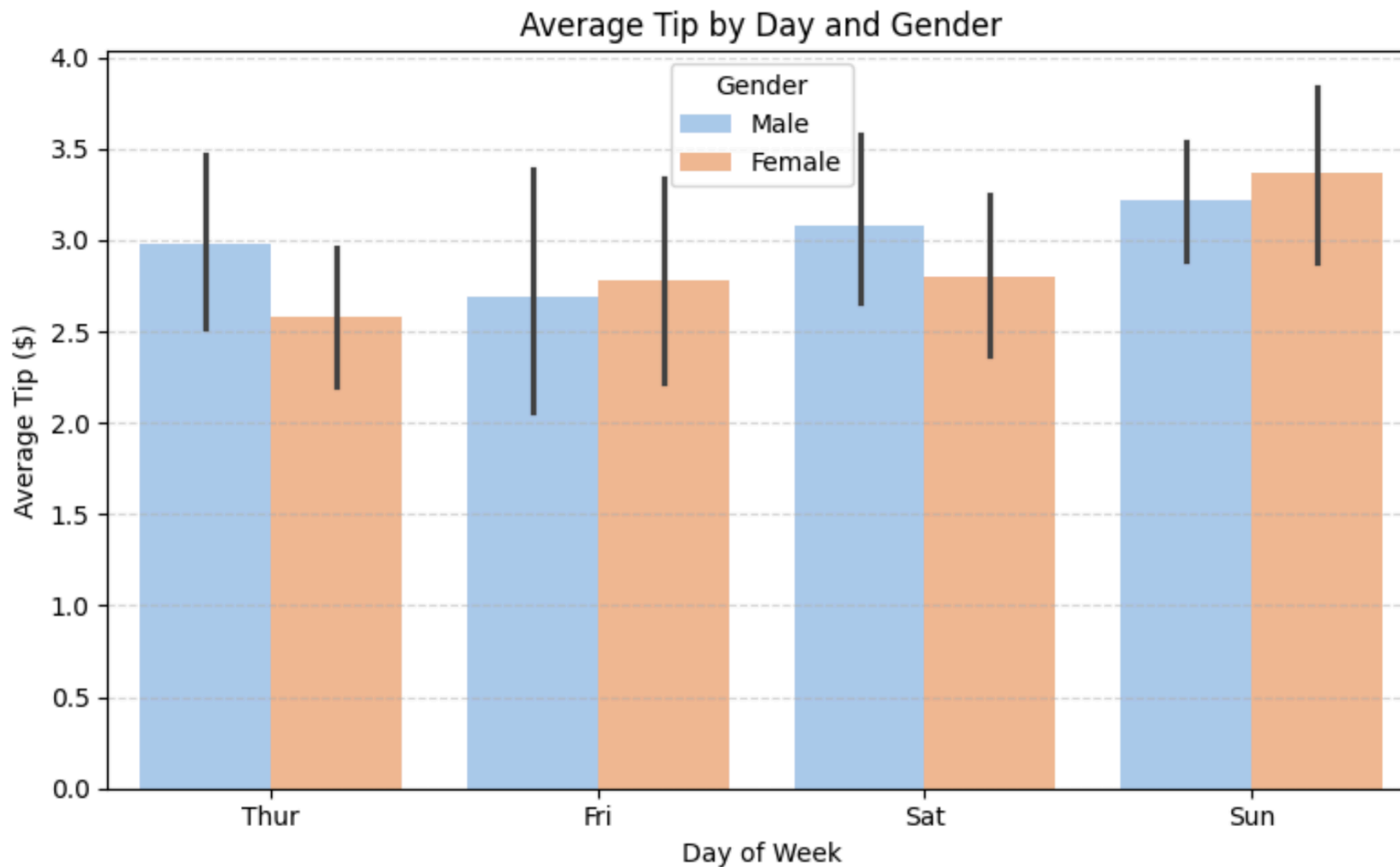
```
plt.ylabel('Average Tip ($)')
```

```
plt.legend(title='Gender', loc=0)
```

```
plt.grid(axis='y', linestyle='--', alpha=0.5)
```

```
plt.tight_layout()
```

```
plt.show()
```



데이터 사전 처리

누락 데이터 처리

- 데이터프레임에서 원소 데이터 값이 종종 누락되는 경우가 있음.
 - 데이터를 파일로 입력할 때 오류, 또는 파일 형식을 변환하는 과정에서 데이터 소실
- 일반적으로 유효한 데이터 값이 존재하지 않는 누락 데이터는 NaN(Not a Number)로 표시
- 누락 데이터가 많아질 수록, 데이터의 품질이 떨어지고 데이터 분석 알고리즘을 왜곡하는 현상이 발생됨.

```
# 데이터프레임 개요  
df.info()
```

deck에 $891 - 688 = 203$
즉, 688개의 누락 데이터

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 891 entries, 0 to 890  
Data columns (total 15 columns):  
#   Column                Non-Null Count  Dtype    
---  -  
0   survived              891 non-null    int64    
1   pclass                891 non-null    int64    
2   sex                   891 non-null    object    
3   age                   714 non-null    float64   
4   sibsp                 891 non-null    int64    
5   parch                 891 non-null    int64    
6   fare                  891 non-null    float64   
7   embarked              889 non-null    object    
8   class                 891 non-null    category   
9   who                   891 non-null    object    
10  adult_male            891 non-null    bool      
11  deck                  203 non-null    category   
12  embark_town           889 non-null    object    
13  alive                 891 non-null    object    
14  alone                 891 non-null    bool      
dtypes: bool(2), category(2), float64(2), int64(4), object(5)  
memory usage: 80.7+ KB
```


- value_counts() 메소드를 이용하여, 누락 데이터 (dropna = False) 확인

```
# deck 열의 NaN 개수 계산하기
```

```
df['deck'].value_counts(dropna=False)
```

```
deck
```

```
NaN      688
```

```
C         59
```

```
B         47
```

```
D         33
```

```
E         32
```

```
A         15
```

```
F         13
```

```
G          4
```

```
Name: count, dtype: int64
```

- missingno 라이브러리를 활용하여, NaN 분포를 시각적으로 확인

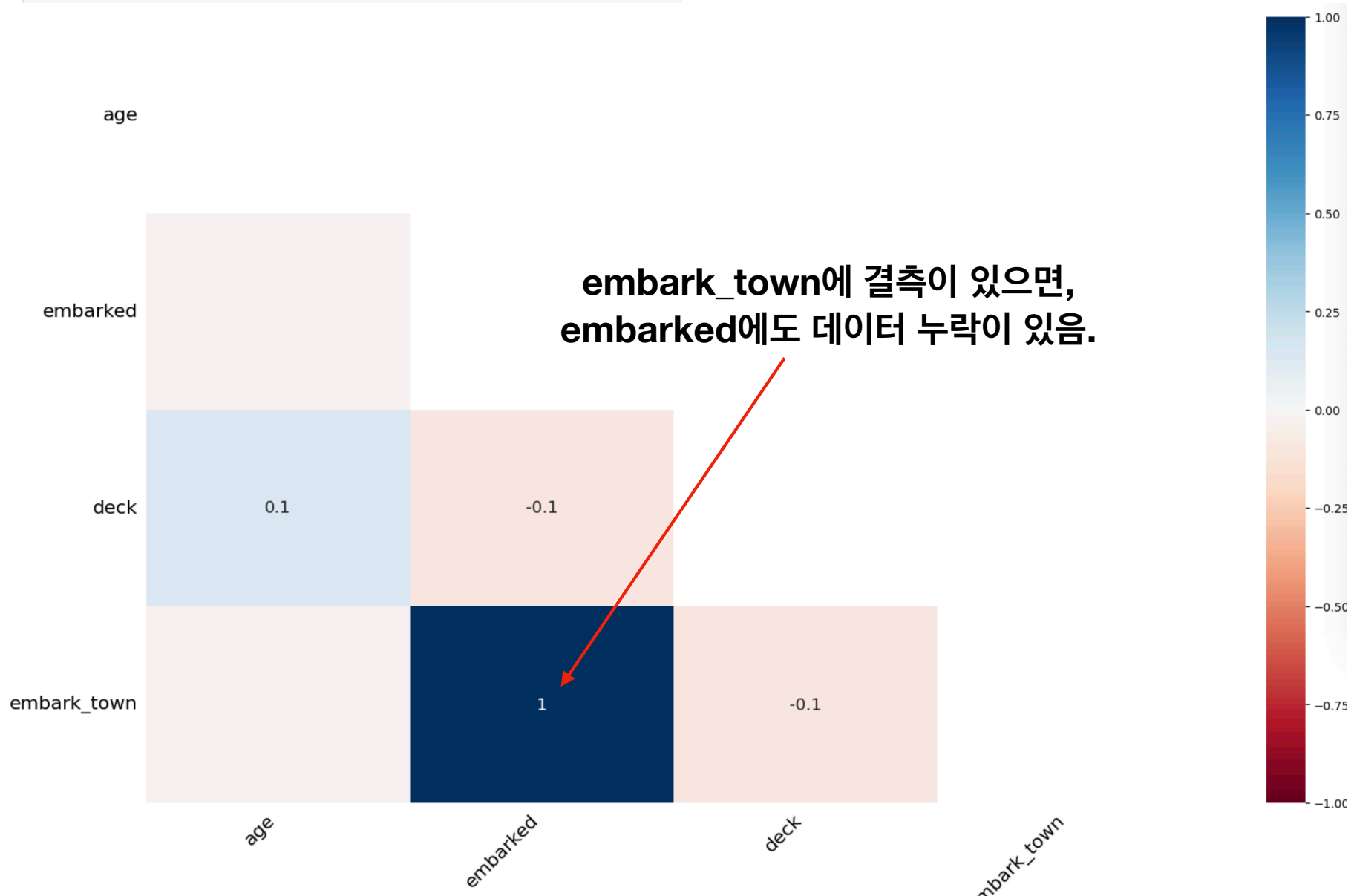
```
# missingno 라이브러리 활용
import missingno as msno
import matplotlib.pyplot as plt

# 매트릭스 그래프
msno.matrix(df)
plt.show()
```



- missingno 라이브러리를 활용하여, 결측치의 상관관계를 분석

```
# 히트맵  
msno.heatmap(df)  
plt.show()
```



- 누락 데이터 처리:
 - 제거 : `df.dropna()`

```
# NaN 값이 500개 이상인 열을 모두 삭제 - deck 열(891개 중 688개의 NaN 값)
```

```
df_thresh = df.dropna(axis=1, thresh=500)
```

```
print(df_thresh.columns)
```

```
df_thresh.head()
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
      'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',  
      'alone'],  
      dtype='object')
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Southampton	no
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	Cherbourg	yes
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no

- 누락 데이터 처리:
 - 제거 : `df.dropna()`

```
# age 열에 나이 데이터가 없는 모든 행을 삭제 - age 열(891개 중 177개의 NaN 값)
```

```
df_age = df.dropna(subset=['age'], how='any', axis=0)
```

```
print(len(df_age))
```

714

```
df_age_fare = df.dropna(subset=['age', 'fare'], how='all', axis=0) # 둘 다 NaN일 때만 제거
```

```
print(len(df_age_fare))
```

891

```
df_age_fare = df.dropna(subset=['age', 'fare'], how='any', axis=0) # 둘 중 하나라도 NaN이면 제거
```

```
print(len(df_age_fare))
```

714

- 누락 데이터 처리: 대체 : `df.fillna()`
 - 데이터의 품질을 높일 목적으로 누락 데이터를 삭제하는 경우, 귀중한 데이터를 완벽하게 활용하지 못하는 경우 발생.
 - 이에 따라, 데이터 중에서 일부가 누락되어 있더라도, 나머지 데이터를 최대한 살려 데이터 분석에 활용하는 경우가 좋을 수 있음.
- 누락 데이터를 대체할 값으로, 데이터의 분포와 특성을 잘 나타낼 수 있는 평균값, 최빈값등을 많이 활용

```
# age 열의 첫 10개 데이터 출력 (5 행에 NaN 값)
df['age'].head(10)
```

```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
5     NaN
6    54.0
7     2.0
8    27.0
9    14.0
```

Name: age, dtype: float64

```
# age 열의 NaN값을 다른 나이 데이터의 평균으로 변경하기
mean_age = df['age'].mean(axis=0) # age 열의 평균 계산 (NaN 값 제외)
df['age'] = df['age'].fillna(mean_age)
```

```
# age 열의 첫 10개 데이터 출력 (5 행에 NaN 값이 평균으로 대체)
df['age'].head(10)
```

```
0    22.000000
1    38.000000
2    26.000000
3    35.000000
4    35.000000
5    29.699118
6    54.000000
7     2.000000
8    27.000000
9    14.000000
```

Name: age, dtype: float64

- `mean()` 메소드에 `skipna=True` 옵션이 기본 적용
 - 만약 `skipna = False` 경우, NaN으로 결과가 표시됨.

```
df = sns.load_dataset('titanic')
mean_age = df['age'].mean(axis=0, skipna = False)
print(mean_age)
```

nan

```
# embark_town 열의 829행의 NaN 데이터 출력
df['embark_town'][825:830]
```

```
825    Queenstown
826    Southampton
827    Cherbourg
828    Queenstown
829             NaN
Name: embark_town, dtype: object
```

```
# embark_town 열의 최빈값(승선도시 중에서 가장 많이 출현한 값) 찾기
most_freq = df['embark_town'].value_counts(dropna=True).idxmax()
print(most_freq)
```

```
# embark_town 열의 최빈값(승선도시 중에서 가장 많이 출현한 값) 찾기
most_freq2 = df['embark_town'].mode()[0]
print(most_freq2)
```

```
Southampton
Southampton
```

항목	mode()[0]	value_counts().idxmax()
설명	최빈값(가장 자주 등장한 값)만 바로 반환	값들의 빈도수 정렬 후 최빈값의 인덱스 반환
가독성	✅ 간단하고 직관적	❌ 상대적으로 복잡함
속도	✅ 빠름	❌ 느릴 수 있음 (전체 분포 계산 필요)
유연성	❌ 단일 최빈값에 적합	✅ 분포 정보 활용
복수 최빈값	첫 번째만 반환 (mode()는 여러 개 가능)	하나만 반환, 여러 개 구분 어려움
NaN 처리	자동 제외 (dropna=True 기본)	옵션으로 지정 가능 (dropna=True/False)
추천 상황	단순 최빈값 추출	빈도 비교

```
# embark_town 열의 NaN값을 최빈값으로 대체하기
```

```
df['embark_town'] = df['embark_town'].fillna(most_freq)
```

```
# embark_town 열 829행의 NaN 데이터 출력 (NaN 값이 most_freq 값으로 대체)
```

```
df['embark_town'][825:830]
```

```
825    Queenstown
```

```
826    Southampton
```

```
827     Cherbourg
```

```
828    Queenstown
```

```
829    Southampton
```

```
Name: embark_town, dtype: object
```

- 누락 데이터 처리: 대체 : df.fillna()

- 데이터셋에 따라, 서로 이웃하고 있는 데이터끼리 유사성을 가질 가능성이 클 때, 앞/뒤 이웃하고 있는 값으로 치환.

```
# embark_town 열의 829행의 NaN 데이터 출력  
df['embark_town'][825:831]
```

```
825    Queenstown
```

```
826    Southampton
```

```
827     Cherbourg
```

```
828    Queenstown
```

```
829         NaN
```

```
830     Cherbourg
```

```
Name: embark_town, dtype: object
```

```
# embark_town 열의 NaN값을 바로 앞에 있는 828행의 값으로  
df1['embark_town'] = df['embark_town'].ffill()  
df1['embark_town'][825:831]
```

```
825    Queenstown
```

```
826    Southampton
```

```
827     Cherbourg
```

```
828    Queenstown
```

```
829    Queenstown
```

```
830     Cherbourg
```

```
Name: embark_town, dtype: object
```

```
# embark_town 열의 NaN값을 바로 뒤에 있는 831행의 값으로  
df2['embark_town'] = df2['embark_town'].bfill()  
df2['embark_town'][825:831]
```

```
825    Queenstown
```

```
826    Southampton
```

```
827     Cherbourg
```

```
828    Queenstown
```

```
829     Cherbourg
```

```
830     Cherbourg
```

```
Name: embark_town, dtype: object
```


중복 데이터 처리

- 동일한 정보를 가진 행이나 레코드가 두 번 이상 반복되는 것.
 - 하나의 데이터셋에서 동일한 관측값이 2개 이상 중복되는 경우, 중복 데이터를 삭제해야 함.

중복 데이터를 갖는 데이터프레임 만들기

```
df = pd.DataFrame({'c1': ['a', 'a', 'b', 'a', 'b'],  
                  'c2': [1, 1, 1, 2, 2],  
                  'c3': [1, 1, 2, 2, 2]})
```

df

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

'''데이터프레임 전체 행 데이터 중에서 중복값
(기본값, keep='first') '''

```
df_dup_first = df.duplicated()  
df_dup_first
```

```
0    False  
1     True  
2    False  
3    False  
4    False  
dtype: bool
```

'''데이터프레임 전체 행 데이터 중에서 중복값 찾기
(keep='last') '''

```
df_dup_last = df.duplicated(keep='last')  
df_dup_last
```

```
0     True  
1    False  
2    False  
3    False  
4    False  
dtype: bool
```

- 특정 열에서 중복 데이터 찾기 (예: c2 열)

	c1	c2	c3	비교 항목	df['c2'].duplicated()	df.duplicated(subset=['c2'])
0	a	1	1	코드 간결성	✓ 더 짧음	약간 더 길지만...
1	a	1	1	의도 명확성	✗ 제한적 (Series 전용)	✓ DataFrame 기준으로 직관적
2	b	1	2	확장성 (여러 열)	✗ 불가능	✓ 가능 (subset=[...])
3	a	2	2	가독성	중간	✓ 더 명확
4	b	2	2			

```
# 데이터프레임의 특정 열 데이터에서 중복값 찾기
col_dup = df['c2'].duplicated()
col_dup
```

```
0    False
1     True
2     True
3    False
4     True
Name: c2, dtype: bool
```

```
# 데이터프레임의 특정 열 데이터에서 중복값 찾기
col_dup2 = df.duplicated(subset=['c2'])
col_dup2
```

```
0    False
1     True
2     True
3    False
4     True
dtype: bool
```

- 중복 데이터 제거

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
df2 = df.drop_duplicates()
df2
```

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
# 데이터프레임에서 중복 행을 제거 (keep='last')
df3 = df.drop_duplicates(keep='last')
df3
```

	c1	c2	c3
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

- keep = False 로 모든 중복 행 제거

```
# 데이터프레임에서 중복 행을 제거 (keep=False)
df4 = df.drop_duplicates(keep=False)
df4
```

	c1	c2	c3
2	b	1	2
3	a	2	2
4	b	2	2

- 중복 데이터 제거

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
# c2, c3열을 기준으로 중복 행을 제거  
df5 = df.drop_duplicates(subset=['c2', 'c3'])  
df5
```

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2

```
# c2, c3열을 기준으로 중복 행을 제거 (중복 데이터를 모두 제거)  
df6 = df.drop_duplicates(subset=['c2', 'c3'], keep=False)  
df6
```

	c1	c2	c3
2	b	1	2

데이터 정규화

- 여러곳에서 수집한 자료들은 단위선택, 대소문자 구분 문제, 약칭 활용등 다양한 형태로 표현될 수 있음.
 - 동일한 대상을 표현하는 방법에 차이가 있으면 분석이 정확도가 매우 낮아짐.
- 데이터 분석에서 데이터 포맷을 일관성 있게 정규화 하는 작업이 매우 중요함.

```
# 열 이름을 지정
df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
              'acceleration', 'model year', 'origin', 'name']

# 데이터프레임의 첫 3행을 출력
df.head(3)
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	name
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	plymouth satellite

- mpg (mile per gallon)을 kpl (kilometer per liter) 로 변환.
변환된 column을 마지막 열로 추가

```
# mpg(mile per gallon)를 kpl(kilometer per liter)로 변환 (mpg_to_kpl = 0.425)
mpg_to_kpl = 1.60934 / 3.78541

# mpg 열에 0.425를 곱한 결과를 새로운 열(kpl)에 추가
df['kpl'] = df['mpg'] * mpg_to_kpl

# 데이터프레임의 첫 3행을 출력
df.head(3)
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	name	kpl
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu	7.652571
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	buick skylark 320	6.377143
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	plymouth satellite	7.652571

- 데이터 종류에 맞게 dtype을 변환

```
print(df.dtypes)
```

```
mpg           float64
cylinders      int64
displacement  float64
horsepower    object
weight        float64
acceleration  float64
model year    int64
origin        int64
name          object
kpl           float64
dtype: object
```

- horse power (마력)은 float이 적절.

```
# horsepower 열의 고유값 확인
```

```
print(df['horsepower'].unique())
```

- CSV 파일을 데이터프레임으로 변환하는 과정에서 문자열로 인식이 되었고, ? 와 같은 특수기호가 포함됨.

```
['130.0' '165.0' '150.0' '140.0' '198.0' '220.0' '215.0' '225.0' '190.0'
 '170.0' '160.0' '95.00' '97.00' '85.00' '88.00' '46.00' '87.00' '90.00'
 '113.0' '200.0' '210.0' '193.0' '?' '100.0' '105.0' '175.0' '153.0'
 '180.0' '110.0' '72.00' '86.00' '70.00' '76.00' '65.00' '69.00' '60.00'
 '80.00' '54.00' '208.0' '155.0' '112.0' '92.00' '145.0' '137.0' '158.0'
 '167.0' '94.00' '107.0' '230.0' '49.00' '75.00' '91.00' '122.0' '67.00'
 '83.00' '78.00' '52.00' '61.00' '93.00' '148.0' '129.0' '96.00' '71.00'
 '98.00' '115.0' '53.00' '81.00' '79.00' '120.0' '152.0' '102.0' '108.0'
 '68.00' '58.00' '149.0' '89.00' '63.00' '48.00' '66.00' '139.0' '103.0'
 '125.0' '133.0' '138.0' '135.0' '142.0' '77.00' '62.00' '132.0' '84.00'
 '64.00' '74.00' '116.0' '82.00']
```

```

# 누락 데이터('?') 삭제
import numpy as np
df['horsepower'] = df['horsepower'].replace('?', np.nan)
df = df.dropna(subset=['horsepower'], axis = 0).copy()

# 이후 안전하게 타입 변환
df['horsepower'] = df['horsepower'].astype('float')

# horsepower 열의 자료형 확인
print(df['horsepower'].dtypes)

float64

```

'?'을 np.nan으로 변경
누락데이터 행을 삭제, ← copy() 추가

문자열을 실수형으로 변환

- dropna()로 행을 제거하면서 원본 DataFrame이 아닌 “복사본(sliced view)”를 반환했을 가능성이 있으므로, copy() 를 명시적으로 사용하여, 완전한 복사본 구성

범주형 데이터 처리

- 구간 분할: 연속 데이터를 구간(bin) 별로 나눠서 분석하는 경우가 효율적일 때.

```
# horsepower 열의 누락 데이터('?') 삭제하고 실수형으로 변환
df['horsepower'] = df['horsepower'].replace('?', np.nan)
df = df.dropna(subset=['horsepower'], axis=0).copy()
df['horsepower'] = df['horsepower'].astype('float')
```

'?'을 np.nan으로 변경
누락데이터 행을 삭제
문자열을 실수형으로 변환

```
# np.histogram 함수로 3개의 bin으로 나누는 경계 값의 리스트 구하기
count, bin_dividers = np.histogram(df['horsepower'], bins=3)
print(bin_dividers)
```

```
[ 46.          107.33333333 168.66666667 230.          ]
```

```
# 3개의 bin에 이름 지정
bin_names = ['저출력', '보통출력', '고출력']

# pd.cut 함수로 각 데이터를 3개의 bin에 할당
df['hp_bin'] = pd.cut(x=df['horsepower'],
                      bins=bin_dividers,
                      labels=bin_names,
                      include_lowest=True)

# 데이터 배열
# 경계 값 리스트
# bin 이름
# 첫 경계값 포함

# horsepower 열, hp_bin 열의 첫 15행을 출력
df[['horsepower', 'hp_bin']].head(10)
```

	horsepower	hp_bin
0	130.0	보통출력
1	165.0	보통출력
2	150.0	보통출력
3	150.0	보통출력
4	140.0	보통출력
5	198.0	고출력
6	220.0	고출력
7	215.0	고출력
8	225.0	고출력
9	190.0	고출력